

The problem: Finding an apartment in Lebanon as a student means navigating scam listings, unpredictable electricity schedules, contracts written to confuse, and no structured way to find a compatible roommate. No existing platform addresses the Lebanese market specifically — its generator dependency, mixed-language contracts, neighbourhood-level electricity variations, and student-centric needs. **NestAI solves all of it in one production-grade AI platform.**

Platform architecture — modular monolith

NestAI is a modular monolith: one deployed application divided into 10 strictly bounded domain modules. Each module owns its code and database tables and communicates with others only through defined service interfaces — never through direct repository imports. This gives clean separation, independent testability, and a clear microservice extraction path without distributed-systems overhead.

MCP (Model Context Protocol) — AI tool layer

MCP connects the LangGraph AI agent to every module's capabilities through a standard protocol. The agent calls 9 tools dynamically: `search_listings`, `calculate_commute`, `get_area_scores`, `check_fraud`, `get_roommate_matches`, `estimate_cost`, `compare_areas`, `transcribe_audio`, `survival_search`. This transforms the agent from a chatbot into an AI operating system.

NestAI

10 domain modules

Module	Responsibility	Key tables	Key MCP tools
Housing	Listings, search, verification	listings, listing_photos, saved_listings	search_listings, calculate_commute
Roommate	Compatibility matching, requests	roommate_requests	get_roommate_matches
Agent	LangGraph orchestration, session	agent_sessions	All 9 MCP tools
Fraud	Scam detection, photo hashing	fraud_reports, listing_photos	check_fraud
Contracts	PDF analyzer, OCR fallback	contracts	analyze_contract
Area Intel	Neighborhood scores	neighborhoods	get_area_scores, compare_areas
Estimator	Lebanon cost breakdown	cost_estimates	estimate_cost
Notifications	SSE, Redis pub/sub	notifications	—
Users	Auth, profiles, onboarding	users, student_profiles	—
Reputation	Landlord reviews + AI summarization	landlord_reviews	—

19

Features

14

Days to build

10

Modules

3

AI tiers

13

Docker services

5

Storage buckets

FeaturesAll 19 features — day built, priority, what it does			
Feature	Day	Priority	What it does
DB schema + seed data	1	Foundation	16 tables, pgvector 1024-dim, 50 Lebanese listings
FastAPI modular monolith + JWT	2	Foundation	10 module folders, student + landlord roles, 15 endpoints
Listings API + landlord portal	3	Foundation	CRUD, phone-verified badge, price-in-range badge
Embedding pipeline + roommate match	4	Core AI	BGE-M3 loads once at worker startup, 1024-dim embeddings
AI relocation agent + MCP	5	Core AI	LangGraph 6-node graph, 9 MCP tools, session state
Fraud + area intelligence	6	Core AI	Price z-score, phone dedup, imagehash phishing
Contract analyzer + OCR fallback	7	Core AI	PyMuPDF → GPT-4o Vision fallback for scanning
Real-time notifications (SSE)	6	Core AI	Redis pub/sub, FastAPI SSE endpoint, no WebSockets
Celery background jobs	2+	Infrastructure	4 queues (high/medium/low/dead), 8 periodic tasks
React onboarding + auth UI	8	Frontend	8-question onboarding feeds all AI features, profile
Listing feed + Leaflet map	9	Frontend	Fraud badges, verification badges, price vs market
Roommate matching UI	10	Frontend	5 dimension progress bars, send/accept/decline
AI agent chat + area dashboard	11	Frontend	Streaming chat, area score bars, compare 2 areas
STT voice input (Whisper)	11	Frontend	MediaRecorder → /agent/transcribe → Whisper
Contract analyzer UI	12	Frontend	PDF upload, loading state, green/yellow/red risk
Landlord dashboard + cost estimator	12	Frontend	Post listings, manage inquiries, Lebanon cost calculator
Demo mode + mobile polish	13	Launch	One-click Lara persona login, profile nudge, mobile
MinIO blob storage + SFTP	2+	Infrastructure	5 typed buckets, magic byte validation, presigned URLs
Full demo rehearsal	14	Launch	Run demo 5x, no new code, prepare pitch and slides

AI Architecture

Agent, memory, RAG, LLM routing, embeddings

LangGraph agent — 6 nodes, 5 checkpoints

Node	Type	What it does	Model tier
parse_intent	LLM	Extracts intent, budget, area, must-haves. Detects creation signals.	Cheap
search_and_rank	Tool caller	Calls search_listings + calculate_commute + get_area_scores + check_fraud	Cheap
validate_results	Checkpoint	Checks: has results, not all shown, fraud threshold. Rule-based up to 2x widening area/budget.	Powerful
compare	LLM	Builds trade-off comparison across top results. Flags mistakes.	Cheap
validate_comparison	Checkpoint	No hallucinated listing IDs, no invented prices. Rule-based on failure.	Powerful
explain_and_respond	LLM	Natural language response. Saves state to agent_session. Fires SSE if needed.	Cheap

3-layer memory architecture

Layer	Technology	TTL / Persistence	What is stored
Short-term	Redis Hash	2 hours per session	messages, shown_listings, filters, retry_count,
Long-term	PostgreSQL	Permanent	student_memory, search_history, liked/rejected
Semantic	pgvector HNSW	Permanent	listing embeddings, profile embeddings, prefer

LLM routing — 3 tiers, every call mapped

Tier	Model	Tasks routed here	Cost/1M tokens
Free	BGE-M3 (local)	embed_listing, embed_profile, embed_query, embed_rag_chunk	\$0.05
Cheap	GPT-4o mini	parse_intent, explain_compatibility, summarize_area, classify_fraud_text, summarize_session, a	\$0.15
Powerful	Claude Sonnet	agent_compare_explain, analyze_contract, ocr_and_analyze_contract, validate_coherence, complex	\$3.20

Fallback chain: Claude Sonnet fails → GPT-4o → GPT-4o mini → stale Redis cache → graceful user-facing message. BGE-M3 is local — never affected by provider outages. **Dynamic escalation:** agent detects complexity (conversation length > 6, compare intent, 3+ listings) and upgrades llm_tier in state automatically. ~85% cost reduction vs using GPT-4o for everything.

RAG pipeline — hybrid retrieval + reranking

Step	Technology	Detail
Ingestion	BGE-M3 + pgvector	4 document types: listings, area intelligence, contract clauses, Lebanese housing FAQ
Retrieval	pgvector + pg_trgm	Hybrid: cosine similarity (semantic) + pg_trgm BM25 (keyword) combined with RRF
Reranking	cross-encoder	ms-marco-MiniLM-L-6-v2 scores query-chunk pairs together. Top 3 chunks to LLM.
Augmentation	Prompt engineering	System + retrieved context + session history + user message. LLM cannot hallucinate listing details
Validation	Agent checkpoint	validate_coherence verifies all cited listing IDs exist in DB before response is sent.

Infrastructure

Redis, Celery, MinIO, SFTP, Docker, CI/CD

Redis — complete key schema (16 patterns)

Key pattern	Type	TTL	Purpose
session:{user_id}:{session_id}	Hash	2h	Agent short-term memory: messages, shown_listings
refresh:{user_id}	String	7d	Hashed refresh token. Deleted on logout or rotation
rate:ip:{ip}:{endpoint}	String	60s	Sliding window rate limiter per IP
rate:llm:{user_id}:{task}	String	24h	Daily LLM call counter per user. Resets midnight
llm:{task}:{prompt_hash}	String	6-24h	Cached LLM responses. TTL varies by task.
embed:{text_hash}	String	48h	Cached BGE-M3 query embedding
sse:{user_id}	PubSub	None	Real-time notification channel per user
lock:embed:{listing_id}	String	30s	Distributed lock — prevents duplicate embeddings
fraud:{listing_id}	Hash	12h	Cached fraud score + evidence list
area:{neighborhood_id}	Hash	24h	Cached area intelligence scores
commute:{listing_id}:{university_id}	String	7d	Cached OSRM commute time in minutes
contract:{contract_id}:status	String	2h	Processing status: pending/ocr_running/analyzed

Celery — 4 queues, 8 workers, 8 Beat tasks

Queue	Concurrency	Tasks	Timeout
nestai:high	4 workers	process_contract_ocr, send_match_notification, notify_fraud_detected	120s
nestai:medium	2 workers	consolidate_session_memory, generate_session_summary, update_preference_vector, run_fraud_analysis	300s
nestai:low	1 worker (1GB)	embed_listing, embed_profile, embed_rag_chunk, hash_embed_seed_data, watch_sftp_uploads	600s
nestai:dead	No workers	Failed tasks after max retries. Monitored via Flower.	Never silently dropped.

MinIO blob storage — 5 typed buckets

Bucket	Max size	Types	Access policy	Lifecycle
listing-photos	5MB/file, 20/listing	jpg, png, webp	Public read, private write	Permanent
contracts	10MB, 20 pages	pdf	Private — presigned 15min GET	Permanent
audio	25MB, 120s	webm, mp3, wav	Private	Auto-delete 24h
avatars	2MB	jpg, png	Public read	Overwrite on upload
doc-uploads	500MB	csv, json, pdf	Service account only	Processed/ or quarantine/

SFTP channel (port 2222): Admin uploads CSV listings, JSON area scores, or PDF documents. Celery Beat `watch_sftp_uploads` runs every 60 seconds, routes by extension to the correct ingestion task. Invalid files quarantined with error JSON. Processed files moved to `processed/` prefix. SSH key auth only — no passwords. **Magic byte validation** on all uploads — never trust Content-Type header.

Docker Compose — 13 services, one command

Service	Image	Port	Purpose
api	nestai-api (FastAPI)	8000	All 10 modules, hot reload in dev, uvicorn in prod
frontend	nestai-frontend (React+Vite)	8000	React + Tailwind + shadcn/ui
db	pgvector/pgvector:pg16	5432	PostgreSQL with pgvector extension, persisted
redis	redis:7-alpine	6379	Cache + Celery broker + pub/sub
minio	minio/minio	9000/9001	S3-compatible object storage + web console
sftp	atmoz/sftp	2222	Admin bulk upload channel, SSH key auth only
worker-high	nestai-api	—	Celery: 4 concurrent, nestai:high queue only, 5GB
worker-medium	nestai-api	—	Celery: 2 concurrent, nestai:medium queue, 512MB
worker-low	nestai-api	—	Celery: 1 concurrent, nestai:low queue, 1GB (Ephemeral)
worker-beat	nestai-api	—	Celery Beat: 8 scheduled periodic tasks
minio-init	minio/mc	—	One-time bucket creation (profile: init)
flower	mher/flower	5555	Celery monitoring dashboard (profile: monitoring)
spec-validator	nestai-spec	—	Spec-Kit validation before every deploy (profile: validation)

Security & Stack

Auth, guardrails, authorization, full technology stack

JWT authentication — full design

Access tokens: 15-minute lifetime, HS256 signed, payload contains sub/role/exp/iat/jti. Stored in React memory — never localStorage (XSS risk). Refresh tokens: 7-day lifetime, stored as HttpOnly cookie (JS cannot read) and hashed in Redis with TTL. On every use: verify Redis presence, issue new access + new refresh, delete old refresh (rotation). On logout: delete from Redis immediately. Passwords: bcrypt work factor 12, never stored or logged in plaintext.

Authorization matrix — 12 endpoints

Endpoint	Anonymous	Student	Landlord
GET /listings	Yes	Yes	Yes
POST /listings	No	No	Yes
PUT /listings/{id}	No	No	Own only
GET /roommate/matches	No	Yes	No
POST /agent/chat	No	Yes	No
POST /contracts/analyze	No	Yes	No
GET /fraud/{id}	No	Yes	Yes
GET /landlord/dashboard	No	No	Own only
POST /reviews/{landlord_id}	No	Yes	No
GET /notifications/stream	No	Yes	Yes
POST /agent/transcribe	No	Yes	No
GET /areas/{name}	Yes	Yes	Yes

AI guardrails — 6 protection layers

Guardrail	Where applied	What it blocks
Prompt injection sanitizer	Before every LLM call	15 known patterns: 'ignore previous instructions', role overrides, system prompt hijacking
Output coherence validator	After explain_and_respond	Hallucinated listing IDs and invented prices — regenerates response if detected
Per-user LLM rate limit	LLM router	50 agent calls/day, 10 contract analyses, 20 transcriptions. Redis counter reset daily
File magic byte validation	Upload endpoints + storage gateway	Checks first 8 bytes — never trusts Content-Type header or file extension
File upload size guard	Per-bucket limits	Photos 5MB, PDFs 10MB, audio 25MB. Rejects before MinIO upload.
Content moderation	Agent input/output	Inappropriate content in messages, especially in roommate request flow

Complete technology stack

Frontend React + Vite + Tailwind + shadcn/ui Component-driven, mobile-first, accessible primitives	Backend FastAPI (Python 3.11) Async-native, auto OpenAPI, modular monolith structure	Database PostgreSQL 16 + pgvector Relational data + 1024-dim vector embeddings in one store
---	---	--

Cache Redis 7 + Celery Session, LLM cache, rate limits, pub/sub, 4-queue async job system	Embeddings BAAI/bge-m3 Free, multilingual (Arabic/French/English), 1024-dim, loaded once at worker start	Agent LangGraph 6-node stateful graph, session memory, MCP tool registry, checkpoints
Cheap LLM GPT-4o mini 9 tasks: intent parse, summaries, compatibility explanations. ~\$0.15/1M tokens	Powerful LLM Claude Sonnet 5 tasks: contract analysis, multi-listing reasoning, coherence validation	OCR GPT-4o Vision + pdf2image NEW Fallback for scanned lease PDFs — same JSON output as text path
STT OpenAI Whisper-1 NEW Voice → text for agent chat. Arabic/French/English. Zero LangGraph changes.	Photo fraud imagehash (phash) NEW Perceptual fingerprint per photo, Celery dedup check across all listings	PDF extract PyMuPDF (fitz) Primary text extraction. Falls back to GPT-4o Vision if empty result.
Blob storage MinIO (S3-compatible) NEW 5 buckets, presigned URLs, magic byte validation, SFTP watcher	Routing OSRM (OpenStreetMap) Real commute time to Lebanese universities. Free, no API cost.	Auth JWT + bcrypt + Redis 15-min access, 7-day HttpOnly refresh with rotation
Real-time Server-Sent Events One-directional push via Redis pub/sub. No WebSocket needed.	Spec Spec-Kit 11 YAML specs validate module implementations before every deploy	Deploy Docker Compose + GitHub Actions 13 services, one command. CI: test → spec-validate → build → SSH deploy
Monitoring Sentry + Flower + structlog Error aggregation, Celery dashboard, JSON structured logs with request IDs		

Database & Timeline

Schema, seed data, and 14-day build plan

Database schema — 16 tables

Table	Module	Key columns
users	Users	id, email, password_hash, role (student/landlord/admin)
student_profiles	Users	university, budget, sleep_schedule, embedding vector(1024), preference_vector vector(1024)
landlord_profiles	Users	phone, phone_verified, reputation_score
neighborhoods	Area Intel	electricity, internet, transport, safety, student_vibe, generator_cost
universities	Housing	name, lat, lng — 10 Lebanese universities seeded
listings	Housing	price, bedrooms, amenities, embedding vector(1024), fraud_score, status
listing_photos	Fraud	minio_key, phash (perceptual hash for dedup)
listing_verifications	Housing	phone_verified, photos_reviewed, price_in_range
saved_listings	Housing	user_id, listing_id (composite PK)
fraud_reports	Fraud	score, evidence JSONB, price_zscore, photo_flags, text_flags
roommate_requests	Roommate	from_user_id, to_user_id, score, dimensions JSONB, status
contracts	Contracts	minio_key, ocr_used, analysis JSONB, status
cost_estimates	Estimator	rent, generator, water, internet, transport, total_monthly
notifications	Notifications	type (enum), payload JSONB, read
landlord_reviews	Reputation	maintenance, responsiveness, honesty, hidden_fees (all 1-5), ai_summary
agent_sessions	Agent	state JSONB, history JSONB, summary (LLM-generated)
student_memory	Agent	preferred_areas, preference_vector vector(1024), liked_count
rag_chunks	Agent	source_type, chunk_text, embedding vector(1024), language

14-day build sprint

Day(s)	Phase	What gets built
1	Foundation	Database schema + 50 seed listings + Docker Compose (13 services) + MinIO
2	Foundation	FastAPI modular monolith + JWT auth (2 roles) + Celery worker setup + API ve
3	Foundation	Listings CRUD API + landlord portal + verification badges + file upload presigne
4	Core AI	BGE-M3 embedding pipeline + roommate compatibility engine (5 dimension sco
5	Core AI	LangGraph agent (6 nodes, 5 checkpoints, 9 MCP tools, session memory, dyna
6	Core AI	Fraud detection (price z-score + phone dedup + imagehash) + area intelligence
7	Core AI	Contract analyzer (PyMuPDF + GPT-4o Vision OCR fallback) + API tests + con
8	Frontend	React + Vite + Tailwind setup + JWT auth UI + 8-question onboarding flow
9	Frontend	Listing feed + Leaflet map + fraud badges + verification badges + save listing
10	Frontend	Roommate matching UI (5 dimension bars) + request flow + side-by-side comp
11	Frontend	AI agent chat + area dashboard + survival quick-actions + Whisper STT mic bu
12	Frontend	Contract analyzer UI (color-coded risk cards) + landlord dashboard + cost estim
13	Launch	Demo mode (one-click Lara persona) + mobile fixes + empty states + productio
14	Launch	Full demo rehearsal x5. No new code. Prepare pitch + 5 anticipated questions.

What NestAI gives every user

What they gain	How NestAI delivers it
Time — hours not days	AI agent narrows 200 listings to 5 relevant options in one conversation
Protection from scams	Fraud detection flags suspicious listings with specific evidence before student w
Local knowledge	Electricity hours, generator costs, internet quality per neighbourhood — data no
Roommate compatibility	Matched on sleep/study/cleanliness/guests/budget — not just availability
Lease clarity	Contract analyzer reads any PDF or scanned paper and flags hidden fees and u
Real monthly cost	Estimator surfaces generator subscription + water delivery + internet — the hido
Continuity across sessions	Agent remembers preferences, liked listings, rejected areas across all visits
Voice-first mobile	Whisper STT lets students speak queries in Arabic, French, or English

Generated codebase — 22 files, 3,637 lines

File	Lines	Purpose
core/config.py	~70	Pydantic Settings — all env vars, dev/prod aware
core/database.py	~60	Async engine (pool_size=10) + sync engine for Celery + get_db dependency
core/redis.py	~90	3 clients (async/sync/pubsub) + RedisKeys class with all 16 key patterns
core/storage.py	~110	MinIO client, 5 buckets, magic byte validation, presigned URLs, ownership check
core/llm_router.py	~130	3-tier routing, TASK_TIERS dict, fallback chain, Redis cache, rate limiting
core/embeddings.py	~90	BGE-M3 service, worker_init signal, embed_text/batch/listing/profile/query
core/celery_config.py	~110	4 queues, all task routes, retry policies, 8 Beat scheduled tasks
core/logging.py	~95	structlog JSON, RequestIDMiddleware, global exception handler
core/security.py	~120	Rate limit middleware, security headers, magic byte validation, prompt injection sanitizer
app/main.py	~65	FastAPI factory, lifespan, Sentry, Redis pool, all 10 routers, health endpoint
migrations/init.sql	~250	All 16 tables + HNSW indexes + pg_trgm + seed universities + seed neighbourhoods
seed/listings.sql	~70	50 realistic Lebanese listings across 8 neighbourhoods with real prices
specs/all_modules.yaml	~270	All 10 module specs + agent spec (6 nodes, 5 validators, 9 MCP tools)
tasks/sftp.py	~130	SFTP watcher, CSV/JSON ingestion, quarantine pattern, area score cache invalidation
tests/test_embeddings.py	~55	Unit tests: dimensions, normalize flag, batch efficiency, profile fields
tests/test_api_listings.py	~120	Integration tests: auth flow, CRUD, ownership, rate limits
tests/test_agent_flow.py	~170	Agent behavior: parse_intent, validate_results retry, full conversation, survival bypass
docker/docker-compose.yml	~160	All 13 services with health checks, resource limits, shared env anchor
docker/deploy.yml	~100	GitHub Actions: test → spec-validate → build → SSH deploy → health check
.env.dev / .env.prod	~70	Complete environment templates for both environments

3-sentence pitch for presentations

NestAI is an AI-powered student housing platform built specifically for Lebanon — where finding an apartment means navigating scam listings, unpredictable electricity, hidden fees, and contracts written to confuse you.

Unlike generic listing sites, NestAI gives every student a personal AI relocation agent that understands their voice, knows their budget, matches them with compatible roommates, analyzes their lease before they sign — even when it is a scanned paper document — and surfaces the real monthly cost including generator subscriptions and water delivery.

It is not a listing website. It is the informed local friend every student needs when moving to a new city.